

Desenvolvimento de uma Biblioteca de Pré-processamento de Dados Utilizando Python

Bruno Sampaio¹, Levi Falcão¹, Moisés Souza¹, Raphael Moreira¹

Centro Universitário de Excelência (Unex)

Caixa Postal S/N – 44085-370 – Feira de Santana – BA – Brazil

{[bruno.silva](mailto:bruno.silva@aluno.unex.edu.br), [levi.queiroz](mailto:levi.queiroz@aluno.unex.edu.br), [moises.oliveira](mailto:moises.oliveira@aluno.unex.edu.br), [raphael.moreira](mailto:raphael.moreira@aluno.unex.edu.br)}@aluno.unex.edu.br

Abstract. *This paper presents the development of a data preprocessing library in Python using only native language resources, without external libraries. The solution operates on datasets represented as dictionaries of lists and implements missing value handling, Min-Max normalization, z-score standardization, Label Encoding and One-Hot Encoding. The library was organized into specialized classes and a facade class named Preprocessing. The results indicate that the implementation is suitable for educational purposes and for understanding essential data preparation steps before machine learning tasks.*

Resumo. *Este artigo apresenta o desenvolvimento de uma biblioteca de pré-processamento de dados em Python utilizando apenas recursos nativos da linguagem, sem bibliotecas externas. A solução opera sobre datasets representados como dicionários de listas e implementa tratamento de valores ausentes, normalização Min-Max, padronização por escore z, Label Encoding e One-Hot Encoding. A biblioteca foi organizada em classes especializadas e em uma classe fachada denominada Preprocessing. Os resultados indicam que a implementação é adequada para fins didáticos e para a compreensão das etapas fundamentais de preparação de dados antes de tarefas de aprendizado de máquina.*

1. Introdução

O pré-processamento de dados é uma etapa fundamental em projetos de ciência de dados, pois dados incompletos, inconsistentes, em escalas distintas ou com atributos categóricos não convertidos podem comprometer análises e modelos. Han, Kamber e Pei (2011) destacam que a preparação adequada dos dados é parte central do processo de mineração de dados.

Na atividade proposta, a Dende Softhouse descreve um cenário em que dados incorretos, incompletos e categóricos dificultam o uso posterior em sistemas de recomendação. Nesse contexto, foi solicitado o desenvolvimento de uma biblioteca em Python, sem o uso de bibliotecas externas, capaz de operar sobre datasets representados como dicionários de listas.

Diante disso, este projeto tem como objetivo desenvolver uma biblioteca educacional de pré-processamento de dados que implemente operações essenciais de tratamento de valores ausentes, escalonamento numérico e codificação de variáveis categóricas.

Além do desenvolvimento da biblioteca, este projeto também busca evidenciar a importância do pré-processamento como etapa indispensável antes do uso de dados em tarefas analíticas e preditivas. Assim, a proposta não se limita à implementação técnica,

mas também à compreensão do papel dessas transformações na melhoria da qualidade e da usabilidade dos dados.

Este artigo está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórica sobre pré-processamento; a Seção 3 detalha a metodologia e a estrutura modular da biblioteca; a Seção 4 discute os resultados obtidos com o dataset do Spotify; e a Seção 5 apresenta as considerações finais.

2. Fundamentação Teórica

O pré-processamento reúne técnicas aplicadas antes da modelagem com o objetivo de melhorar a qualidade da base de dados. Entre essas técnicas, destacam-se o tratamento de valores ausentes, o escalonamento de atributos numéricos e a codificação de variáveis categóricas.

2.1 Valores Ausentes ou Nulos

Valores ausentes são comuns em bases reais e podem surgir por falhas de coleta ou ausência de preenchimento. Little e Rubin (2019) apontam que a forma como esses valores são tratados influencia diretamente a análise. Entre as estratégias mais utilizadas estão a remoção de registros incompletos e a substituição por valores definidos.

A Figura 1 apresenta um exemplo ilustrativo de uma base contendo valores ausentes. No tratamento desses dados, podem ser utilizadas diferentes operações. A operação *isna* permite identificar registros com valores ausentes, retornando apenas as linhas incompletas. De forma complementar, *notna* seleciona apenas os registros válidos, ou seja, aqueles que não possuem valores nulos.

Além disso, a operação *fillna* realiza a substituição dos valores ausentes por um valor definido, como 0, permitindo manter a estrutura do dataset. Por outro lado, a operação *dropna* remove completamente os registros que apresentam valores ausentes, reduzindo o conjunto de dados, porém garantindo a ausência de inconsistências.

artista	followers
<i>A</i>	None
<i>B</i>	1200
<i>C</i>	None
<i>D</i>	450

Figura 1. Exemplo de valores ausentes e nulos. Fonte: Elaborado pelos autores (2026).

2.2 Escala e Normalização

No caso de atributos numéricos, diferenças de escala podem prejudicar algoritmos de aprendizado. Por isso, são utilizadas técnicas como a normalização Min-Max, que converte os valores para um intervalo fixo, e a padronização por escore z, que recentraliza os dados com base na média e no desvio padrão.

valor original	Min-Max	Z-score (aprox.)
50	0.0	-1.22
75	0.5	0.00
100	1.0	1.22

Figura 2. Exemplo de normalização. Fonte: Elaborado pelos autores (2026).

Como ilustrado na Figura 2, a aplicação da normalização Min-Max permite reduzir diferenças de escala entre atributos, tornando-os comparáveis. Por sua vez, a padronização por escore z é útil quando se deseja analisar a posição relativa dos valores em relação à média da distribuição.

2.3 Encoders

Já variáveis categóricas precisam ser convertidas para formato numérico antes do uso em muitos algoritmos. O Label Encoding associa cada categoria a um número inteiro, enquanto o One-Hot Encoding cria colunas binárias para representar cada categoria separadamente.

album_type	album	single	compilation
album	1	0	0
single	0	1	0
compilation	0	0	1

Figura 3. Exemplo de Label e One-Hot encoding. Fonte: Elaborado pelos autores (2026).

Como ilustrado na Figura 3, o *Label Encoding* converte categorias em valores inteiros, enquanto o *One-Hot Encoding* cria novas colunas binárias para cada categoria observada.

3. Metodologia

A solução foi desenvolvida em Python com uso exclusivo de recursos nativos da linguagem, conforme exigido pela atividade. O dataset é tratado como um dicionário em que cada chave representa uma coluna e cada valor corresponde a uma lista com os registros daquela coluna. Essa organização foi adotada com o objetivo de tornar a solução mais modular e facilitar a separação de responsabilidades entre as operações implementadas. Com isso, a biblioteca se torna mais compreensível do ponto de vista didático e mais simples de utilizar em diferentes etapas do fluxo de preparação dos dados.

A biblioteca foi organizada em quatro classes principais. A classe *MissingValueProcessor* concentra as operações de identificação e tratamento de valores ausentes. A classe *Scaler* implementa os métodos de normalização e padronização. A classe *Encoder* realiza a codificação de variáveis categóricas. Por fim, a classe *Preprocessing* atua como fachada, centralizando o acesso às operações da biblioteca.

Antes da aplicação das transformações, a biblioteca valida se todas as colunas possuem o mesmo número de elementos, garantindo consistência estrutural do dataset.

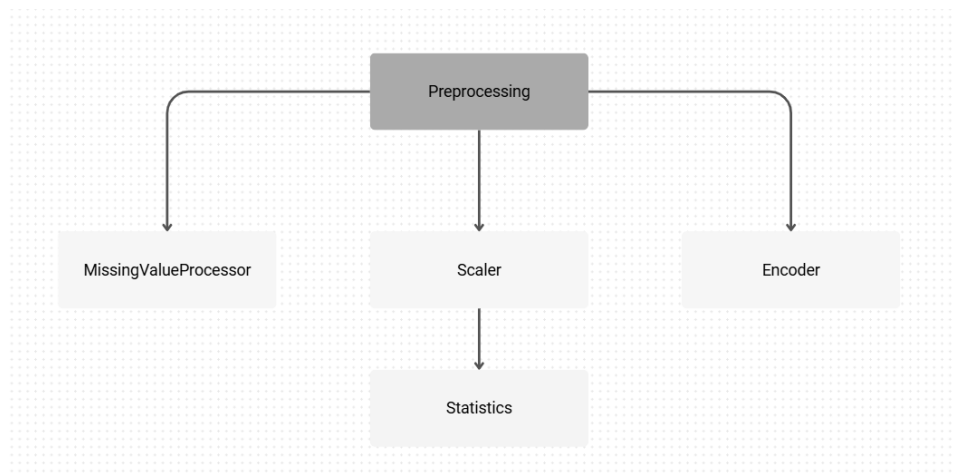


Figura 4. Arquitetura da Biblioteca. Fonte: Produção dos autores (2026)

3.1 Teste

A validação da biblioteca foi realizada por meio do script isolado, utilizando um *dataset* sintético projetado especificamente com casos de borda (valores nulos, diferentes escalas numéricas e categorias textuais).

A verificação de sucesso ocorreu via impressão no console, confirmando se as funções executam a limpeza, o escalonamento e a codificação corretamente. O script validou tanto a precisão dos cálculos matemáticos quanto a integridade estrutural do dicionário, assegurando que a biblioteca operasse de forma consistente e livre de falhas de execução. Na figura 5, podemos ver o script retornando uma mensagem de sucesso.

```
[Running] python -u "c:\Users\Bruno\Documents\preprocessamento\teste_preprocessing_original.py"
.....
-----
Ran 12 tests in 0.005s

OK
```

Figura 5. Validação da biblioteca através do script de testes. Fonte: Captura de tela direta do software pelos autores (2026).

4. Resultados e Discussão

A biblioteca foi aplicada ao dataset [Spotify Songs for ML and Analysis](#), conforme solicitado na atividade. A análise foi conduzida com o objetivo de observar o comportamento das operações implementadas sobre uma base real.

Inicialmente, foram identificados valores ausentes nas colunas selecionadas. Em seguida, aplicaram-se estratégias de preenchimento e/ou remoção desses registros, conforme a necessidade do experimento. Posteriormente, variáveis numéricas foram transformadas por normalização ou padronização, reduzindo diferenças de escala entre colunas. Por fim, atributos categóricos foram convertidos para representação numérica por meio de Label Encoding e One-Hot Encoding.

ANTES		DEPOIS	
Musica	Seguidores	Musica	Seguidores
A	None	A	0
B	25.784	B	25.784
C	None	C	0
D	47.785	D	47.785

Figura 6. Exemplo de utilização do *fillna* na coluna seguidores. Fonte: Elaborado pelos autores (2026).

```
[Running] python -u "c:\Users\Bruno\Documents\preprocessamento\minmax.py"
```

artist_popularity	artist_popularity_scaled	artist_followers	artist_followers_scaled
77	0.77	2812821	0.02
64	0.64	2363438	0.02
48	0.48	193302	0.0
77	0.77	2813710	0.02
48	0.48	8682	0.0

Figura 7. *Min-Max* nas colunas *artist_popularity* e *artist_followers*. Fonte: Captura de tela direta do software pelos autores (2026).

```
[Running] python -u "c:\Users\Bruno\Documents\preprocessamento\encode.py"
```

LABEL ENCODING	
album_type	album_type_label
album	0
single	2
single	2
album	0
single	2

ONE-HOT ENCODING			
album_type	album_type_album	album_type_compilation	album_type_single
album	1	0	0
single	0	0	1
single	0	0	1
album	1	0	0
single	0	0	1

Figura 8. *Label Encoding* e *One-Hot Encoding* no dataset do Spotify. Fonte: Captura de tela direta do software pelos autores (2026).

Na figura 6, foram considerados valores ilustrativos, pois no dataset do Spotify não ocorre None nas colunas. Após a aplicação da operação *fillna*, na coluna *artist_followers*, esses valores foram substituídos por 0, garantindo consistência na coluna.

Já as variáveis *artist_popularity* e *artist_followers*, apresentadas na figura 7, originalmente em escalas distintas — sendo a primeira no intervalo [0, 100] e a segunda variando em ordem de milhões — foram normalizadas utilizando o método Min-Max.

Após a transformação, ambas passaram a assumir valores no intervalo [0, 1], permitindo maior comparabilidade entre os atributos.

Na codificação categórica, apresentada na figura 8, foi utilizada a coluna `album_type`. Por meio do Label Encoding, cada categoria foi convertida em um identificador numérico inteiro. Em seguida, aplicou-se o One-Hot Encoding, gerando colunas binárias correspondentes a cada categoria observada, como `álbum`, `single` e `compilation`.

Os métodos utilizados foram: *fillna* (MissingValueProcessor), *Min-Max* (Scaler), *Label Encoding* e *One-Hot Encoding* (Encoder). Foram apresentados pelo menos um método de cada classe para fins didáticos.

4.2 Implementação dos métodos *isna* e *notna*

Esta subseção detalha a lógica de código das funções de identificação de dados ausentes da classe MissingValueProcessor, evidenciando a manipulação direta sobre a estrutura de dicionário de listas.

4.2.1 *Isna*

O método *isna* tem como objetivo retornar apenas os registros que possuem valores ausentes nas colunas analisadas. Inicialmente, a função define as colunas que serão verificadas por meio do método auxiliar `_get_target_columns`. Em seguida, cria um novo dicionário de saída com a mesma estrutura do dataset original, porém com listas vazias para armazenar apenas as linhas filtradas. Depois disso, o método percorre cada linha do dataset e utiliza uma variável booleana para verificar se existe algum valor ausente nas colunas-alvo. Essa verificação não se limita apenas ao valor `None`: ela é realizada pelo método auxiliar `_is_missing`, que também considera como ausentes strings vazias ou representações textuais como `"n/a"`, `"na"`, `"null"` e `"none"`.

```
def isna(self, columns : Set[str] = None) → Dict[str, List[Any]] :
    target_columns = self._get_target_columns(columns)
    all_columns = list(self.dataset.keys())
    result_dataset = {}
    for col in all_columns :
        result_dataset[col] = []
    if not all_columns :
        return result_dataset
    row_count = len(self.dataset[all_columns[0]])
    for row_index in range(row_count) :
        has_missing = False
        for col in target_columns :
            if self._is_missing(self.dataset[col][row_index]) :
                has_missing = True
        if has_missing :
            for col_all in all_columns :
                result_dataset[col_all].append(self.dataset[col_all][row_index])
    return result_dataset
```

Figura 9. Código final do método *isna*. Fonte: Elaborado pelos autores (2026).

4.2.2 Notna

O método *notna* atua de forma complementar ao *isna*, retornando apenas os registros que não apresentam valores ausentes nas colunas selecionadas. Assim como no método anterior, a função começa definindo as colunas-alvo e preparando um novo dicionário com a mesma estrutura do dataset original. Em seguida, o algoritmo percorre cada linha da base e verifica, por meio da função `_is_missing`, se existe algum dado ausente nas colunas analisadas. Se nenhum valor ausente for encontrado, a linha é considerada válida e seus valores são copiados integralmente para o dicionário de saída.

```
def notna(self, columns : Set[str] = None) → Dict[str, List[Any]] :
    target_columns = self._get_target_columns(columns)
    all_columns = list(self.dataset.keys())
    result_dataset = {}
    for col in all_columns :
        result_dataset[col] = []
    if not all_columns :
        return result_dataset
    row_count = len(self.dataset[all_columns[0]])
    for row_index in range(row_count) :
        has_missing = False
        for col in target_columns :
            if self._is_missing(self.dataset[col][row_index]) :
                has_missing = True
        if not has_missing :
            for col_all in all_columns :
                result_dataset[col_all].append(self.dataset[col_all][row_index])
    return result_dataset
```

Figura 10. Código final do método *notna*. Fonte: Elaborado pelos autores (2026).

5. Conclusão

Este projeto apresentou o desenvolvimento de uma biblioteca de pré-processamento de dados em Python construída apenas com recursos nativos da linguagem. A solução implementa operações essenciais de tratamento de valores ausentes, escalonamento numérico e codificação de variáveis categóricas, organizadas em uma estrutura modular com classes especializadas e uma classe fachada.

A atividade contribuiu para a compreensão prática dos fundamentos do pré-processamento de dados, demonstrando como essas transformações podem ser implementadas manualmente e aplicadas a uma base real. Como projetos futuros, sugere-se ampliar a biblioteca com tratamento de duplicatas, preenchimento automático por média e mediana e novas estratégias de transformação de dados.

6. Referências

HAN, J.; KAMBER, M.; PEI, J. Data Mining: Concepts and Techniques. 3. ed. Waltham: Elsevier, 2011.

LITTLE, R. J. A.; RUBIN, D. B. Statistical Analysis with Missing Data. 3. ed. Hoboken: Wiley, 2019.